

OLYMPUZ

Olympuz Coder

Complete Manual

English Edition

Version 1.0.0 · PRD revisions: Studio 2026.07.05 / Ops 2026.07.05 / Marketing 2026.07.06

A multi-department AI coding agent · Author: Patriarch

Contents

1. Introduction
2. Installation & Requirements
3. Architecture — The Department Pattern
4. Studio — Design & Generation Department **STUDIO**
5. Ops — Agentic Operations Department **OPS**
6. Marketing & Growth Department **MARKETING**
7. Safety, Approval Gates & Kill Switches
8. Configuration (settings.json)
9. Environment Variables
10. Built-in Specialist Agents
11. Tools Reference
12. Knowledge Graph & the Curator
13. Workflows & Quick Start
14. Troubleshooting
15. Command Quick Reference

1. Introduction

Olympuz Coder is an AI coding agent (a fork of Claude Code) extended with three specialist **departments** that turn it from a single assistant into an on-demand organization. Each department is a self-contained "team" of purpose-built agents that share a common engine, inject their own Product Requirements Document (PRD) into the system prompt, plan work as a dependency graph, and curate reusable knowledge to a knowledge graph.

Department	Default	What it does
STUDIO Design & Generation	Enabled	Design systems & design-token generation for web, Android and Windows; brand-aligned UI/UX, design-token emission (CSS, Compose Kotlin, WinUI XAML).
OPS Agentic Operations	Opt-in	Computer-use control of Windows & any browser; vision, speech-to-text (ears), text-to-speech (voice); autonomous agent teams with an org board.
MARKETING Marketing & Growth	Enabled	Complete marketing campaigns: image / video / short-form generation, copywriting, official profiles, social-media publishing, send & receive email.

The unifying promise. Every department **plans before it acts, asks a human before anything outward-facing or destructive**, and **logs what it publishes** so it can be reviewed and reversed. You stay in control; the agents do the heavy lifting.

2. Installation & Requirements

2.1 System requirements

- **OS:** Windows 11 (primary), macOS or Linux also work.
- **Node.js:** 22+ (the SQLite knowledge-graph backend uses the built-in `node:sqlite`; Bun uses native `bun:sqlite`).
- **RAM:** 6 GB minimum practical; the launcher auto-clamps the V8 heap to 55% of free RAM, range `[768, 3072]` MB, to avoid swap-thrash freezes on low-RAM boxes.
- **Disk:** ~600 MB for dependencies + browser binaries if you use Ops.

2.2 Install & build

```
# from the repository root
npm install          # install dependencies
npm run build        # bun esbuild bundle → dist/cli.mjs
npm run smoke        # build + verify the binary boots (prints version)

# optional gates
npm run typecheck    # tsc --noEmit (must report 0 errors)
npm test             # full vitest suite (~2639 pass | 0 fail | 7 skip)
```

2.3 Run it

```
# interactive TUI
node dist/cli.mjs

# one-shot prompt (headless)
node dist/cli.mjs -p "explain this repository"

# show version
node dist/cli.mjs --version      # → 1.0.0 (Olympuz Coder)
```

The `olympuz` launcher (`bin/olympuz` / `olympuz.cmd` on Windows) wraps `dist/cli.mjs` and sets the dynamic heap. Source-mode `npm run dev` exists but is not a supported production workflow — always run the built bundle.

3. Architecture — The Department Pattern

Every department is a sibling under `src/<dept>/` and follows the same proven shape. A small **shared core** (`src/departments/`) wires them together.

3.1 Inside a department

Module	Responsibility
<code>types.ts</code>	Config, intent, governance, role, plan, run-result types.
<code>principles.ts</code>	The department PRD (full + compressed). A versioned grounding document injected into the system prompt when active.
<code>intent.ts</code>	Detects whether a user message is a request for this department, with a confidence score and triggers.
<code>governance.ts</code>	Policy resolution with the hierarchy Director > domain > default . Admin-settable keys per domain.
<code>department.ts</code>	The specialist org chart (roles + missions) and the planner that turns intent + governance into a dependency-ordered task graph.
<code>curator.ts</code>	The single module that touches the knowledge graph — extracts reusable assets after a run and persists them under a namespaced key.
<code><dept>Engine.ts</code>	Lazy singleton: holds config + governance; exposes <code>is<Dept>Active()</code> , intent detection, planning, curator runner.
<code>inject.ts</code>	<code>compose<Dept>AppendSystemPrompt(existing)</code> — appends the PRD when active, returns input unchanged when dormant.

3.2 The shared core (`src/departments/`)

File	Responsibility
<code>inject.ts</code>	<code>composeAllDepartmentInjections(existing)</code> stacks Studio + Ops + Marketing prompts in sequence. Each early-returns when inactive, so the stack is byte-identical to "before" when no department is active.
<code>approval.ts</code>	<code>buildApprovalCheck</code> / <code>needsApproval(policy, destructive)</code> — the gate primitive used by all outward-facing tools. Policies: <code>allow</code> , <code>ask-destructive</code> , <code>ask-always</code> .
<code>publishedLog.ts</code>	Append-only JSONL of everything published (Marketing). <code>recordPublish</code> , <code>listPublishes</code> , <code>getPublish</code> , <code>markReversed</code> .

Test-safety invariant. `isTestEnv()` is true under vitest (`process.env.VITEST` or `NODE_ENV` `=== 'test'`). Every department is **dormant** under test regardless of config, so prompts, agent lists and tool lists stay byte-for-byte stable. The full suite of ~2639 tests is unaffected by the departments.

4. Studio — Design & Generation Department STUDIO

PRD v2026.07.05 · Default: enabled · Activation: auto-on for build intent.

Studio is the design department. On detecting design/build intent it produces a design system and emits concrete **design tokens** for three platforms from one source of truth, plus a delegation plan for the specialist roster.

4.1 Supported surfaces

- **Platforms:** `web` , `android` , `windows` .
- **Aesthetics (moods):** `neutral-adaptive` , `luxury-dark-gold` , `calm` , `vibrant` , `minimal` , `bold` .
- **Token outputs:** CSS custom properties (web), Compose Kotlin (`Color.kt` / `Type.kt` / `Shapes.kt` , Android), WinUI XAML `ThemeDictionaries` (Windows).
- **Quality gates:** WCAG contrast verification (`contrastAA`), semantic color roles (bg / fg / accent), a verified base color.

4.2 The `/studio` command

```
/studio                                # help
/studio run <brief>                    # plan + tokens for a brief
/studio tokens [base] [mood] [--platform web|android|windows]
/studio principles                      # print the full Studio PRD
/studio admin list                     # show active governance
/studio admin get <domain> <key>       # resolve one policy
/studio admin set <domain>.<key>=<val> # set a policy
/studio knowledge stats                 # knowledge-base counts
/studio knowledge recall <query>       # recall studio knowledge
/studio enable | disable                # toggle the department
/studio status                         # current state
```

4.3 Example session

```
/studio run a calm fintech dashboard for web, luxury mood
/studio tokens #0b1f3a luxury-dark-gold --platform android
```

5. Ops — Agentic Operations Department OPS

PRD v2026.07.05 · Default: **opt-in (disabled)** · Activation: explicit `/ops enable` or `OLYMPUZ_OPS_ENABLED`.

Ops turns Olympuz into a computer-use agent. It controls Windows and any browser, sees the screen (vision), hears (STT), speaks (TTS), and can form autonomous agent **teams** coordinated through a native swarm runtime with an org board.

Ops is opt-in for safety. Real machine control is powerful and dangerous, so Ops is dormant until you explicitly turn it on. **Destructive actions always ask for per-action approval** (mouse/keyboard, shell, file-delete, browser submit/payment). Read-only actions — screenshot, the accessibility tree, vision, listen, speak, read-only browse — do not ask.

5.1 Surfaces

Surface	Capability
<code>computer</code>	Windows desktop control via PowerShell + .NET (<code>System.Drawing</code> screenshot, <code>System.Windows.Forms</code> cursor/keystroke, UIAutomation a11y tree).
<code>browser</code>	Deterministic Playwright when selectors are known; vision-driven when not. Navigation confined to a governance allowlist.
<code>vision</code>	Screenshot metrics + multimodal LLM judgment (describe, OCR, compare).
<code>voice</code>	STT (streaming or batch; project keyterm boosting) and TTS (<code>speak()</code> : Windows SAPI / macOS <code>say</code> / Linux <code>espeak</code> / provider).
<code>automation</code>	Multi-step plan: idempotent step graph, fail-safe, step budget (<code>automation.maxSteps</code>).
<code>teams</code>	Autonomous teammates via native swarm (<code>TeamCreate</code> , <code>SendMessage</code> , <code>Agent</code>). <code>teams.requireApproval</code> gates creation; <code>teams.maxTeammates</code> bounds size.

5.2 The `/ops` command

```
/ops                                # help
/ops run <task>                     # plan + delegate a control task
/ops team list                       # show autonomous-team capability
/ops team new <goal>                 # plan a team for a goal
/ops team spawn <role>               # plan spawning one specialist
/ops review                          # recent ops knowledge + state
/ops admin list | get <domain> <key> | set <domain>.<key>=<val>
/ops knowledge stats | recall <query>
```



```
/ops principles           # print the full Ops PRD
/ops enable | disable    # toggle (opt-in)
/ops status               # current state
```

5.3 Example session

```
/ops enable
/ops run open the browser and find the cheapest flight to Lisbon
/ops team new triage my inbox daily at 9am
```

6. Marketing & Growth Department MARKETING

PRD v2026.07.06 · Default: enabled · Activation: auto-on for campaign/creative/social/email intent.

Marketing builds complete campaigns end-to-end: strategy, copy, on-brand visual creative, video and short-form generation, official profile creation, social-media publishing across five platforms, and full email lifecycle (send + receive).

The publish gate (non-negotiable). Every **publish / send / post** asks a human for approval **before** it goes out, and is then appended to the reversible **published log**. You can review everything published and reverse what isn't right: `/marketing review`, `/marketing reverse <id>`. Read actions (inbox) never ask.

6.1 Capabilities

- **Creative generation:** images (ImageGen — DALL·E 3 / Stability / local), video & shorts (VideoGen — Veo 3 / Sora 2 / Runway Gen-5 / Kling 3).
- **Channels:** X, Meta, LinkedIn, TikTok, YouTube (publish + delete); Email via SMTP + IMAP (send + inbox).
- **Compliance built in:** email is permission-only, double opt-in, with a clear unsubscribe path.

6.2 The `/marketing` command

```
/marketing                # help
/marking run <brief>       # plan + delegate a campaign
/marketing review [channel] # see everything published
                           #   channel: x|meta|linkedin|tiktok|youtube|email
/marketing reverse <id>    # reverse a published social post
/marketing admin list | get <domain> <key> | set <domain>.<key>=<val>
/marketing knowledge stats | recall <query>
/marketing principles      # print the full Marketing PRD
/marketing enable | disable # toggle the department
/marketing status          # current state
```

6.3 Example session

```
/marketing run launch a campaign for Olympuz Coder on X and LinkedIn
/marketing review                # audit what was published
/marketing reverse ab12cd34      # reverse a specific post
```

6.4 Configuring the adapters

Adapters are **no-op-when-unconfigured**: when credentials are absent they return a clear `{ ok: false, error: "configure X" }` rather than throwing, so the deterministic core is always testable. Live operation needs the relevant env vars set (see §9).

Adapter	Library	Env keys (summary)
Video	<code>fetch</code> to provider	<code>VEO_API_KEY</code> / <code>VEO_API_URL</code> , <code>SORA_*</code> , <code>RUNWAY_*</code> , <code>KLING_*</code>
Email	nodemailer (SMTP) + imapflow (IMAP)	<code>SMTP_HOST/USER/PASS/ ...</code> , <code>IMAP_*</code>
Social	<code>fetch</code> per channel	<code>X_API_*</code> , <code>META_*</code> , <code>LINKEDIN_*</code> , <code>TIKTOK_*</code> , <code>YOUTUBE_*</code>

7. Safety, Approval Gates & Kill Switches

Safety is structural, not advisory. Three independent mechanisms keep you in control.

7.1 The approval gate

Every outward-facing or destructive tool implements `checkPermissions(input, context)` which returns one of:

- `{ behavior: 'allow', updatedInput }` — proceeds.
- `{ behavior: 'ask', message, updatedInput }` — pauses and shows you exactly what it wants to do; you approve or deny.

The policy is resolved from governance with the hierarchy **Director > domain > default**:

Policy	Behavior
<code>ask-always</code>	Asks every time. <i>Marketing's publish actions use this.</i>
<code>ask-destructive</code>	Asks only for destructive actions. <i>Ops' default for computer/browser control.</i>
<code>allow</code>	Proceeds without asking. Not recommended for outward-facing actions.

7.2 The published log (Marketing)

Append-only JSONL at `<CLAUDE_CONFIG_DIR>/marketing/published.jsonl`. Every published item records: `id`, `channel`, `action`, `target`, `content`, `timestamp`, `status`, `reversible`, and a `reversalHint`. Reversal deletes/unlists the post and marks the entry `reversed`. (Email sends are generally not mechanically reversible — send a correction/follow-up instead.)

7.3 Kill switches (absolute)

Each department's `OLYMPUZ_<DEPT>_ENABLED=false` is a **hard kill switch**. It is checked **first**, before config and before the test force-active hatch. Setting it disables the department even if config says otherwise.

Switch	Scope
<code>OLYMPUZ_OPS_ENABLED=false</code>	Ops fully off — no PRD injection, no ops-* agents/tools, no machine control.
<code>OLYMPUZ_MARKETING_ENABLED=false</code>	Marketing fully off — no PRD injection, no marketing-* agents/tools, no publishing.
<code>OLYMPUZ_COMPUTER_CONTROL</code>	Gates the legacy ComputerControlTool (defense in depth).

OLYMPUZ_DEVICE_BRIDGE	Gates the ADB device bridge.
------------------------------	------------------------------

8. Configuration (settings.json)

Each department has an optional block in your settings schema. Settings are loaded from `~/olympuz/settings.json` (or the platform equivalent). The slash commands (`/studio enable` , etc.) mutate the in-memory config at runtime.

8.1 Studio

```
"studio": {
  "enabled": true,
  "autoActivate": true,
  "intensity": "ultra",           // "standard" | "premium" | "ultra"
  "defaultPlatform": "web",      // "web" | "android" | "windows"
  "aesthetic": "neutral-adaptive" // see moods in §4.1
}
```

8.2 Ops

```
"ops": {
  "enabled": false,              // OPT-IN (defaults false)
  "autoActivate": false,
  "intensity": "standard",
  "surfaces": ["computer", "browser", "vision", "voice", "automation"],
  "approvalPolicy": "ask-destructive" // "ask-destructive" | "ask-always" | "allow"
}
```

8.3 Marketing

```
"marketing": {
  "enabled": true,              // mirrors Studio
  "autoActivate": true,
  "intensity": "ultra",
  "channels": [],               // allowlist; empty = all
  "publish": {
    "approvalPolicy": "ask-always", // recommended
    "logging": true                // append to published log
  }
}
```

8.4 Governance directives

Set any policy from the chat with `/<dept> admin set <domain>.<key>=<value>` . For Director-level overrides use the `director.<domain>.<key>` form, which beats every other source. Booleans accept `true/false` ; comma-separated values become lists only when a comma is present (a single value stays a string).

9. Environment Variables

Variable	Purpose
<code>OLYMPUZ_OPS_ENABLED</code>	<code>false</code> = Ops hard kill switch (absolute).
<code>OLYMPUZ_MARKETING_ENABLED</code>	<code>false</code> = Marketing hard kill switch (absolute).
<code>OLYMPUZ_COMPUTER_CONTROL</code>	Gates the legacy ComputerControlTool.
<code>OLYMPUZ_DEVICE_BRIDGE</code>	Gates the ADB device bridge.
<code>OLYMPUZ_NATIVE_CORE</code>	Gates the optional native core.
<code>OLYMPUZ_LOCAL_MODEL</code>	Local model name for the local inference provider.
<code>OLYMPUZ_LOCAL_ENDPOINT</code>	Local model endpoint URL.
<code>OLYMPUZ_LOCAL_AUTO_PULL</code>	Auto-pull the local model if missing.
<code>CLAUDE_CONFIG_DIR</code>	Override the config directory (also locates the published log).
<code>NODE_OPTIONS</code>	The test suite sets <code>--max-old-space-size=8192</code> via this (NOT a node flag — vitest workers don't inherit parent V8 flags).

Provider/channel credentials (SMTP/IMAP, X/Meta/LinkedIn/TikTok/YouTube tokens, Veo/Sora/Runway/Kling keys) are also read from the environment; see §6.4.

10. Built-in Specialist Agents

Each department registers its specialist roster only when active; they appear as `subagent_type` values for the `Agent` tool. Every specialist carries its department's PRD + safety bar in its system prompt.

10.1 Ops roster (10 roles)

Role (subagent_type)	Mission (summary)
<code>ops-director</code>	Admin of admins. Owns task scope, the safety boundary, sign-off.
<code>ops-computer-operator</code>	Desktop control via the computer-use loop; routes destructive actions through the gate.
<code>ops-browser-operator</code>	Playwright + vision browser automation; confined to the allowlist.
<code>ops-vision</code>	Eyes. Screenshot metrics + multimodal judgment.
<code>ops-ears</code>	STT. Streaming/batch transcription with keyterm boosting.
<code>ops-voice</code>	TTS. <code>speak()</code> — concise, language-aware.
<code>ops-automation-engineer</code>	Designs the idempotent step graph; honors <code>automation.maxSteps</code> .
<code>ops-team-lead</code>	Forms + coordinates autonomous teammates (native swarm).
<code>ops-sentinel</code>	Safety reviewer. Adversarially rejects plans with any gap.
<code>ops-curator</code>	Persists reusable macros/selectors/workflows to the knowledge base.

10.2 Marketing roster (9 roles)

Role (subagent_type)	Mission (summary)
<code>marketing-director</code>	Admin of admins. Owns objective, positioning, the publish gate, sign-off.
<code>marketing-strategist</code>	Objective + audience + measurable goal; channel mix; funnel stage.
<code>marketing-copywriter</code>	Brand-voice copy. Hero → proof → one CTA. AIDA/PAS/FAB.
<code>marketing-designer</code>	On-brand visual creative (ImageGen); variants; never misleading.
<code>marketing-videographer</code>	Video / shorts (VideoGen). Hook < 1s, payoff, CTA, captions, ≤60s.
<code>marketing-social-manager</code>	Publishes to social; confined to <code>social.channels</code> ; every post approved + logged.

<code>marketing-email-manager</code>	Email lifecycle; permission-only, double opt-in; every send approved + logged.
<code>marketing-analytics</code>	Instruments from day one; attributes; kills what doesn't move the goal.
<code>marketing-curator</code>	Mines finished campaigns for reusable assets; persists to the knowledge base.

Studio exposes its own specialist roster (design-system lead, UX, UI, brand, motion, accessibility, curator). Inspect the live roster with `/studio run` or `/studio status`.

11. Tools Reference

Department tools are registered only when their department is active. All outward-facing tools route through the approval gate (§7.1).

Tool	Dept	Purpose & gating
<code>StudioBuild</code>	Studio	Plan + emit design tokens. Read-only.
<code>OpsBuild</code>	Ops	Plan + delegate a control task. Read-only.
<code>BrowserControl</code>	Ops	Playwright adapter. <code>submit</code> / <code>fill</code> ask approval per <code>browser.scope</code> .
<code>Vision</code>	Ops	Wraps <code>analyzeImage</code> / <code>analyzeScreenshot</code> / <code>judgeWithModel</code> . Read-only.
<code>Speak</code> / <code>Listen</code>	Ops	TTS / STT wrappers. Read-only.
<code>ComputerControl</code>	(legacy)	Gated by <code>OLYMPUZ_COMPUTER_CONTROL</code> . Defense in depth.
<code>MarketingBuild</code>	Marketing	Plan + delegate a campaign. Read-only.
<code>VideoGen</code>	Marketing	Generate video/shorts. Internal (not outward-facing) — no approval.
<code>Email</code>	Marketing	<code>send</code> (approval + log) / <code>inbox</code> (read, no approval).
<code>SocialPost</code>	Marketing	<code>post</code> / <code>short</code> (approval + log) / <code>delete</code> (approval + marks log reversed).
<code>ImageGen</code>	(shared)	DALL-E 3 / Stability / local; provider-gated.

12. Knowledge Graph & the Curator

Each department's **curator** is the only module that writes to the knowledge graph. After a run it extracts reusable assets and persists them under a namespaced key, so departments never collide and lessons compound over time.

Department	Entity types
Studio	<code>studio-*</code> (design systems, token sets, brand assets)
Ops	<code>ops-macro</code> (recorded action sequences), <code>ops-selector</code> (winning selectors), <code>ops-workflow</code>
Marketing	<code>marketing-campaign</code> , <code>marketing-creative</code> (+prompt), <code>marketing-copy</code> , <code>marketing-audience</code>
Marketing publish	<code>marketing-publish</code> (one entity per published item — the audit trail)

Persistence uses `node:sqlite` under Node (22+) and `bun:sqlite` under Bun, behind a unified adapter. `shutdownGlobalGraph()` closes all cached provider handles — important on Windows to avoid `EPERM` on directory cleanup. Recall any department's knowledge with `/<dept> knowledge recall <query>`.

13. Workflows & Quick Start

13.1 Design a product (Studio)

```
/studio run a calm productivity app for Windows, minimal mood
# → tokens emitted for web + Android + Windows; delegation plan for the roster
```

13.2 Automate a desktop task (Ops)

```
/ops enable
/ops run open the browser, log the cheapest USB-C hub price to prices.csv
# → destructive steps (save file) ask approval; read-only steps (browse) do not
```

13.3 Ship a campaign (Marketing)

```
/marketing run launch Olympuz Coder on X and LinkedIn with one hero image
# → strategy + copy + creative + publish-ready items; each post asks before it goes live
/marketing review                # audit everything published
/marketing reverse <id>          # undo a post that isn't right
```

13.4 Daily operations checklist

1. `/<dept> status` — confirm the department you need is active.
2. `/<dept> run <brief>` — review the plan before delegating.
3. Approve outward-facing actions when prompted.
4. `/<dept> knowledge recall <query>` — reuse prior learnings.
5. For Marketing: `/marketing review` regularly; reverse anything wrong.

14. Troubleshooting

Symptom	Fix
Department seems inactive	Run <code>/<dept> status</code> . Check <code>enabled</code> and the kill switch env var. Ops requires explicit <code>/ops enable</code> .
"configure X" result from an adapter	The relevant credentials aren't set. Add the env vars (\$6.4, \$9) and restart.
Tool/agent absent	Department tools/agents register only when active. Activate the department, then retry.
Freezes after several steps (low-RAM box)	The launcher clamps the heap to <code>[768, 3072] MB @ 55% free RAM</code> . Close other apps; rerun <code>olympuz</code> so it recomputes free RAM.
Suite flakes under load	On 6 GB boxes a few dynamic- <code>import()</code> tests can time out under memory pressure. They pass green in isolation; rerun the specific file.
<code>EPERM</code> on Windows cleanup	Knowledge-graph handles left open across tests. <code>shutdownGlobalGraph()</code> closes them; ensure it's called in <code>afterAll</code> .
Source-mode <code>npm run dev</code> crashes	Not a supported workflow. Always run the built <code>dist/cli.mjs</code> .

15. Command Quick Reference

```
# Shared subcommand shape across all three departments:
/<dept> run <brief>                # plan + delegate
/<dept> status                      # current state
/<dept> enable | disable           # toggle
/<dept> principles                 # print the PRD
/<dept> admin list|get|set         # governance (Director > domain > default)
/<dept> knowledge stats|recall    # curated knowledge

# Studio-only
/studio tokens [base] [mood] [--platform web|android|windows]

# Ops-only (opt-in)
/ops team list|new <goal>|spawn <role>
/ops review

# Marketing-only
/marketing review [x|meta|linkedin|tiktok|youtube|email]
/marketing reverse <id>

# Hard kill switches (absolute, checked first)
OLYMPUZ_OPS_ENABLED=false
OLYMPUZ_MARKETING_ENABLED=false
```

Olympuz Coder v1.0.0 — Complete Manual (English). PRD revisions: Studio 2026.07.05, Ops 2026.07.05, Marketing 2026.07.06. Generated from the source tree of `C:\OLYMPUZ` (branch `feat/multiagent-harden`).